

RegTechBench: Evidence-Aware Benchmarks for Blockchain Regulatory Monitoring

Jeongmin Kim
appff13@gmail.com

Abstract

Blockchain applications increasingly need regulatory monitoring systems that screen transactions, flag risky patterns, respond to sanctions lists and incident reports, and document regulatory reports. Yet existing public datasets do not provide a benchmark unit for these systems: they may contain transactions, graph traces, market records, or copied categories, but they usually do not state what evidence was available, when it arrived, where each category came from, and which response the evidence can support.

We address this gap by designing six workload families for regulatory and risk-control evaluation on blockchain transaction systems, and by building a reproducible dataset package for those workloads. A workload specifies the replayed entries, usable evidence, category meaning, and measured response. The six families cover recent-history rules, repeated-transfer graph patterns, delayed external risk entries, public-source translation, response footprint, and evidence freshness. The package combines generated test cases with converted samples from public anti-money-laundering (AML) and non-fungible token (NFT) datasets. It contains 413,531 transaction-like entries, 384 summaries for delayed or incorrect external risk entries, and 3.38M same-token NFT back-and-forth matches.

The package also ships deterministic reference checks that make each workload executable. These checks show when missing evidence changes the claim being measured. On the same rolling-cap workload, a rule that sees only the current transaction reaches 0.167 precision because it lacks account history. A rule that keeps rolling history reaches 0.863 precision. A paired-cap rule remains at 0.186 precision because the available fields still do not express the full account-pair condition. The result is an auditable benchmark package that lets researchers, protocol teams, and compliance operators compare methods while keeping policy requirements, available evidence, category source, response scope, and timing separate.

1 Introduction

Regulated or risk-managed blockchain services increasingly need to make operational decisions while transactions continue to settle on public ledgers. A service may need to screen a transfer before it is accepted, send an account to manual review, report suspicious activity, block a transfer, or quarantine related accounts after an incident report marks an address as risky. This pressure is visible in official guidance, risk assessments, enforcement guidance, and market-abuse research. FATF guidance treats virtual-asset service providers as subject to risk-based anti-money laundering and counter-terrorist financing obligations and emphasizes that the virtual-asset sector is fast moving and technically dynamic [1]. OFAC has issued virtual-currency-specific sanctions compliance guidance for firms that must screen, report, block, or otherwise respond to sanctions exposure [2]. The U.S. Treasury has also published risk assessments for decentralized finance and NFTs, identifying illicit-finance, sanctions, fraud, theft, and market-integrity risks that motivate transaction monitoring, reporting workflows, sanctions screening, and incident-response policies [3, 4]. Together, these sources point to the same systems problem: regulated or risk-managed blockchain services need ways to evaluate controls while transaction systems keep operating.

Evaluating these controls requires more than a transaction file and a positive/negative label. A report can be written after an event has already happened, while a real-time block requires the necessary evidence before the transfer is executed. A review action may include only a directly labeled account, while a quarantine action may include related accounts or transactions. An evaluator, such as a regulatory monitoring system or compliance replay harness, must therefore know which evidence was available, when it arrived, where each category came from, and what response that evidence can support.

Existing datasets answer important but different tasks. Blockchain data releases often focus on raw chain extraction, crime-category classification, or domain-specific market traces. XBlock-ETH releases processed Ethereum transactions, contracts, and token data for broad analysis [7]. The Elliptic Bitcoin dataset and Elliptic2 provide Bitcoin transaction graphs or subgraphs for illicit-activity classification [6, 8], and BitcoinHeist studies ransomware-related Bitcoin transactions [9]. AMLNet motivates publicly shareable AML-style transaction benchmarks, but it remains generated financial-transaction data rather than public-chain ground truth [11]. EX-Graph, Dune, and

NFT-70M provide useful NFT graph or market sources, but the package uses them under narrower meanings: sender-receiver graph evidence, pattern-filtered source categories, and bounded market traces without abuse answers [12, 10, 13]. These sources are valuable, but they do not by themselves provide a reusable benchmark unit for regulatory and risk-control tasks.

The challenge is that these controls are not ordinary single-entry classification tasks. Some ask whether one transfer is above a limit. Some ask whether recent transfers from the same account exceed a cap. Some ask whether accounts and assets form a repeated-transfer pattern. Some ask how a system should respond when information from outside the chain arrives after earlier transfers have already settled. The meaning of a category also changes by source: a generated answer can be exact for the generator’s rule, a public-dataset category is inherited from another task, and a trace without an answer can test replay behavior but not precision or recall. Collapsing these cases into one accuracy number would hide the actual evaluation claim.

We address this gap by designing six workload families for testing blockchain regulatory monitoring systems and by building a reproducible dataset package for those workloads. We call each repeatable evaluation task a *workload*. A workload states which transfer entries are replayed, which extra evidence the method can use, which condition the method is supposed to check, and which output or response is being measured. The six families cover recent-history rules, repeated-transfer graph patterns, delayed external risk entries, public-source translation, response footprint, and evidence freshness. The package combines generated workloads, generated summaries for delayed or incorrect external risk entries, and converted samples from AMLNet, EX-Graph/Dune, and NFT-70M. It gives researchers controlled workloads for concrete systems questions: which fields are enough for a rule, when graph context is needed, what changes when external risk entries arrive late, and which accounts or transactions become candidates for a response.

This paper makes three contributions:

- **A six-family workload design for testing blockchain regulatory monitoring systems.** The workload families capture recent-history rules, repeated-transfer graph patterns, delayed external risk entries, public-source translation, response footprint, and evidence freshness. Each family states the evidence it requires, the source of any category, the timing of that evidence, and the response that can be evaluated.
- **A reproducible workload package built from generated and public sources.** The package maps generated workloads, AMLNet, EX-Graph/Dune, and a bounded NFT-70M market sample into a common replay format while keeping generated answers, copied public categories, external risk entries, unlabeled traces, and redistribution limits separate.
- **Deterministic reference checks that expose claim boundaries.** The shipped checks show when missing history changes a result, when public categories agree or mismatch with a checked condition, when graph topology can be queried without price or timing claims, and when delayed risk entries or broader quarantine change response coverage.

2 Workload Design Goals

The package treats blockchain regulatory and risk-control evaluation as a data and evidence problem. The test is not only about a detector; it is also about the inputs, external risk entries, rule state, and response attached to the detector output. Two systems can run the same detector but support different responses if they have different evidence or different arrival times for external information. This setting leads to three design goals.

First, a workload must state its evidence assumptions. A threshold rule may need only transaction amount and time. A rolling-window rule needs previous transactions in the same window. A wash-trade or circular-trade rule may need asset identity, transfer direction, and a graph window. A rule that depends on an external risk entry needs the risk category and the time at which that entry became available. If these inputs are implicit, a detector can appear to work only because the package silently gave it evidence that a real deployment may not have.

These evidence requirements change the evaluation claim. A current transfer from account A to account B with amount 60 is enough for a per-transfer cap, but not for a daily cap that also depends on A’s earlier transfers, a bilateral cap that groups transfers by counterparty, or a rule that depends on whether an external risk entry was available before action time. Without those fields, the experiment is not measuring the stated rule.

Second, the package must separate label provenance. A single positive/negative column is too ambiguous for this domain. Generator answers are exact for the generated condition. Public-source categories are inherited from another task. External list or event entries may arrive after relevant transfers have already occurred. Traces without answers support ingestion, distribution, replay, and signal search, but not precision or recall. The package stores

Table 1. Workload families with short examples. A category label attaches a category to a transaction, account, asset, or pattern. Generated labels are exact only for the generator’s rule; labels copied from public sources preserve the category used by the source dataset but are not legal conclusions; external labels arrive from an external list or event seed. The examples show package-entry meaning, not legal facts.

ID	Family	Evidence needed	Short example
W1	Recent-history rules	Transaction amount, time window, account or entity id	<code>tx(w1.001, 2026-01-01T00:10Z, u01>u02, USD, 60); label(oracle, temporal, positive)</code>
W2	Graph patterns	Transfer graph, asset id, time window, address or entity links	<code>tx(w2.014, 2026-01-01T00:20Z, a01>a02, nft7, 1); label(topology, graph, reciprocal)</code>
W3	External risk entries	External risk entry, arrival time, affected transfer endpoint	<code>label(l3.001, address, a01, supplied, high_risk, label.time=00:30Z)</code>
W4	Public-source translation	Source schema, conversion mapping, meaning of any copied category	<code>tx(amlnet.42, 2026-01-01T00:40Z, acct7>acct9, USD, 125); label(proxy, structuring)</code>
W5	Response footprint	Detection output plus configured response action	<code>result(run5, method=quarantine.1hop, target=a01, scope_tx=12, collateral_tx=8)</code>
W6	Evidence freshness	Evidence update time, action time, governance or operator delay	<code>event(w6.003, tx.time=00:05Z, label.time=00:30Z, action=retroactive_review)</code>

these cases as different objects rather than merging generated violations, copied categories, delayed risk entries, and unlabeled traces into one label column.

Third, the package must preserve source and license limits. Many useful public datasets have non-commercial or share-alike licenses. The package therefore distributes project-generated data directly, but provides source notes, checksums, summaries, and rebuild scripts for public sources whose raw rows are not redistributed. Users who obtain the upstream data under its own terms can rerun the conversion without the package treating all sources as having the same license status.

The package therefore combines generated data with samples converted from public datasets. Generated data provides known answers; public data provides realistic transaction shapes. Keeping them separate lets the package support controlled checks while exposing where those checks do or do not align with public traces.

3 Workload Model

The package separates three questions that are often mixed together: what kind of task is being tested, which typed entries represent transactions, categories, results, timing facts, and source data, and what each category is allowed to mean. Table 1 answers the first question using a small record vocabulary, defined below.

We use *record* as the general unit in the package: one typed entry that a user can inspect, replay, or regenerate. A *transfer record* describes a transaction-like event with sender, receiver, asset, amount, and time. A *category record* attaches a category to a transaction, account, asset, or pattern. A *result record* stores the output of a simple check or response simulation. A *timing event record* stores timing facts that are not ordinary transfers, such as when a category became available. A *source record* refers to an upstream dataset entry before conversion. The normalized input is the transfer-format entry that the package feeds to a tested rule or detector in timestamp order.

With this vocabulary, Table 1 lists the six workload families, the information each family needs, and a short example of the package entry shape. Source coverage and category meaning are reported separately in Tables 2 and 3.

The short examples in Table 1 use four forms. A `tx(...)` entry stores id, replay time, sender, receiver, asset, and amount. A `label(...)` entry attaches a category to a target and records why the target is labeled: **oracle** for generator answers, **proxy** for public-source categories, **supplied** for external feeds or event seeds, and **topology** for graph relations. A `result(...)` entry stores the output of a check or response simulation. An `event(...)` entry stores timing facts such as transfer time, label update time, and the response still supportable after the category arrives. These examples abbreviate the public schemas while preserving the distinction among transaction evidence, category meaning, result accounting, and timing.

Category provenance determines which metrics are meaningful. An AMLNet category supports comparison with a converted workload, but remains a source-dataset category. EX-Graph/Dune NFT sender-receiver data supports graph replay, but not price-volume or real-time claims. NFT-70M market traces support ingestion and scan workloads, but the sampled rows have no labeled wash-trading answer. A generated workload with a known answer can support precision and recall against the generator answer; an unlabeled trace cannot.

In practice, the package contains four main typed entries. Transfer entries store the sender, receiver, asset, amount, time, source dataset, and workload id. Category entries store the target, where the category came from,

when it became available if timing matters, what evidence supports it, and a note explaining what the category does not mean. Result entries store the run, method, metric, value, config hash, and output path. A source manifest stores where a source came from and which license or checksum constraints apply.

Transfer entries are replay events, not signed blockchain transactions. They can be fed into a detector, converted into an event stream, or mapped into a contract-test harness. The package standardizes the evidence and category meaning, not the user’s implementation. The JSON schemas are included with the package preparation files.

The schema also supports extension. A new dataset can be added without changing the workload organization if it maps into the same transfer, category, and result schemas. A new workload can reuse existing transfers if it introduces category or result files with explicit meaning. This property is needed because evidence sources, incident labels, NFT markets, and response policies change over time.

4 Workload Construction

Generated and public-source material become the six workload families in Table 1 through a staged construction. Figure 1 gives the construction stack, Table 2 states source coverage by workload family, and Table 3 explains public-source conversion. The package preserves field and category provenance rather than presenting a flat transaction file.

The stack starts from two kinds of material. Generated test cases provide deterministic transfers, categories, and results with known answers for controlled checks. Public-source samples provide external shapes: AMLNet transfer categories, EX-Graph/Dune NFT sender-receiver links, and NFT-70M market traces. Those public sources are useful precisely because they are incomplete in different ways. The construction keeps those gaps visible instead of hiding them behind a single positive/negative label.

Figure 1 abstracts this construction process. Field selection and normalization choose the source fields that can be replayed, rename them into the common package vocabulary, and map source identifiers to deterministic virtual addresses when needed. Evidence augmentation then records the information that a workload needs but a plain transfer row would not carry: generator answers, copied source categories, source provenance, label times, delayed-risk settings, and explicit notes when a source lacks an answer. Workload record assembly turns those pieces into typed package records such as transfer records, category records, result records, timing records, and source records. Context reconstruction finally rebuilds the history, graph neighborhood, external-risk timeline, or response scope that a workload family needs before it can be run as W1–W6. Appendix A expands the same stack into the concrete source fields, row builders, normalized rows, derived processing steps, and package checks used by the current release.

Table 2 is a source-coverage matrix. The *Source input* and *Scale* columns state what exists in the package. The W1–W6 columns state which workload family can use that source input. The cell labels state the kind of support: exact or motif answers from the generator, copied or topology-only categories from public sources, scan-only market traces without answers, source-to-package maps, and generated scope or timing summaries.

The matrix also explains why the package combines generated and public data. Generated workloads provide exact answers for controlled rules, but they do not show how those shapes appear in public datasets. AMLNet, EX-Graph/Dune, and NFT-70M provide public-source shapes, but they do not provide complete legal or abuse ground truth for all workload families. External-risk summaries isolate timing, source-quality, response-scope, and evidence-freshness effects that are hard to observe cleanly in public data. The total row is therefore an inventory of package entries, not one homogeneous dataset.

Table 3 details the public-source branch of Figure 1. It lists the workload family, the public data used, the missing evidence that prevents the source from becoming full ground truth, and the package fields added or preserved during replay conversion.

Together, Tables 2 and 3 set up the source-specific descriptions below. Each public source contributes one kind of evidence: sender-receiver links without price, categories copied from another task, or prices and timestamps without an abuse answer. The package stores the upstream source entry alongside the derived normalized input so a reviewer can audit where a field came from, where a field was generated for replay, and where a copied category should be treated only as approximate evidence.

The scale choices prioritize executability, auditability, and clear category meaning over row count. For generated data, a generator configuration is a fixed recipe for producing a task: it fixes the rule shape, inserted pattern, parameter values, and output files. A generator seed is the fixed value used by the generator’s pseudo-random choices. In this release, the generated-row split is 9 configurations $\times$ 5 seeds, producing 45 deterministic seed runs. Five configurations are recent-history or temporal checks: daily rolling cap, monthly rolling cap, risk exposure threshold, velocity anomaly, and volume-spike anomaly. Four configurations are graph-pattern checks: wash-trade

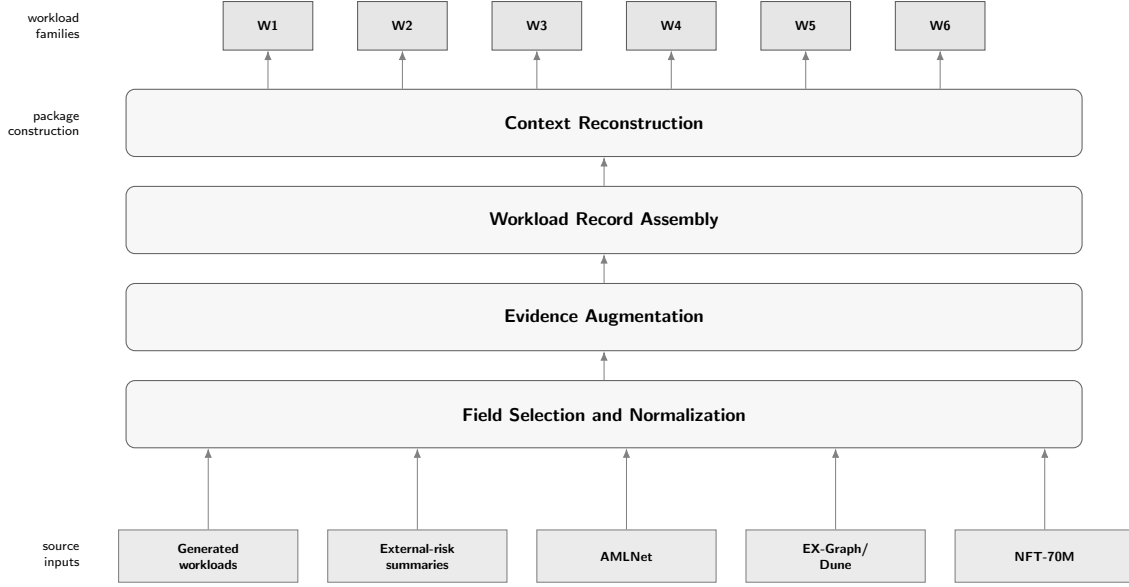


Figure 1: Abstract construction stack for the workload package. The package starts from generated and public-source bundles, normalizes their fields, augments them with explicit evidence and category information, assembles workload records, reconstructs the context needed by each task, and exposes the result as W1–W6 workload families.

motif, structuring pattern, layering/path pattern, and circular-trade motif. These recipes provide controlled W1 and W2 checks, while external-risk summaries cover W3/W5/W6 and public-source conversions cover W4. Running each recipe with different seeds varies addresses, timestamps, amounts, and output order while keeping each run exactly reproducible. For public-source material, the package bounds each sample rather than trying to mirror the full upstream dataset: AMLNet uses 100,000 converted entries, EX-Graph/Dune uses the staged sender-receiver entries available in the current package, and NFT-70M uses seven 1,000-entry samples from different offsets. The rest of this section describes each source input in turn.

Generated entries with known answers. The generated portion is the main distributable part of the package. It contains deterministic entries for rolling transfer caps, graph patterns, structuring, layering, risk exposure, circular trades, velocity anomalies, and volume spikes. The outputs include transfer entries, category entries, detection outputs, and a summary.

These entries are not intended to imitate every public-chain distribution. They provide generator-created answers when public data lacks the context needed for controlled testing. A public trace may contain amount and timestamp but not the account tier, verification status, or configured cap answer needed for a rolling-cap check. The generator supplies those attributes under a fixed configuration, giving each check a known answer.

Delayed external-risk entries use the same controlled design. Public feeds resemble sanctions entries, high-risk address lists, incident seeds, or internal risk lists, but they rarely isolate one variable. The generated test fixes the transaction generator and varies arrival delay, false entries, account coverage, and quarantine radius. In the current package, this workload covers five seeds, 270,000 transfers, and 384 summaries.

The generator also supplies negatives under the same schema. Rare-event tasks need background transfers to measure collateral review, false positives, and workload volume. The package keeps background transfers together with injected positives and distributes the generated entries directly under a permissive project license.

AMLNet-derived workload. We use AMLNet because it is a large public, transaction-like source with sender and receiver identifiers, amounts, timestamps, and AML categories supplied by the source dataset [11]. Those fields make it useful for testing whether transfer-pattern tasks can be rewritten into the package’s common transfer format. The relevant categories are structuring, layering, velocity, and volume. Structuring means splitting activity into smaller transfers, layering means moving value through paths of accounts to make the source harder to follow, velocity means frequent transfers within a time window, and volume means the total amount transferred within a time window.

Table 2. Source coverage by workload family. Workload IDs refer to Table 1. Each nonempty cell states the support provided by the source input. Counts describe packaged files and converted samples, not full upstream public datasets.

Source input	Scale	W1	W2	W3	W4	W5	W6	Category and package meaning
Generated workloads	45 seed runs; 13,829 transfers; 2,043 labels	exact	motif	–	–	–	–	Generator-created answers; distributed directly
External-risk summaries	5 seeds; 270,000 transfers; 1,920 seed-level runs; 384 combined summaries	–	–	delay/noise	–	scope	timing	Generated external-risk scenarios with controlled delay, noise, coverage, and quarantine radius
AMLNet	100,000 converted transfers	copied	copied	–	map	–	–	AML categories copied from source; raw source data is not redistributed
EX-Graph/Dune	22,702 converted entries; 22,551 source-marked transactions; 3,379,930 derived back-and-forth matches	–	topology	–	map	–	–	Sender-receiver source categories; no price or block-time ground truth in the staged source
NFT-70M	Seven 1,000-entry samples; 7,000 converted market entries	–	scan	–	map	–	–	No wash-trading answer; supports ingestion, distribution, and historical scans, not precision or recall
Total entries	413,531 transfers or transaction-like entries	W1–W6 collectively; excludes derived match counts and raw upstream data not redistributed						Inventory total, not a homogeneous dataset

Cell labels. Exact means a generator-created answer for a controlled rule. Motif means a generated graph or path answer. Copied means a category preserved from the upstream source. Topology means sender-receiver graph structure without price or block-time claims. Scan means a trace scan with no abuse answer. Map means conversion into the common package schema. Scope and timing are generated response-coverage and evidence-timing scenarios.

AMLNet is not blockchain ground truth. Its categories come from a synthetic financial-transaction dataset, and its records use payment-style accounts and financial metadata rather than public-chain addresses, assets, and ledger timestamps. A category name such as velocity or volume also does not guarantee a one-to-one match with the sender, receiver, amount, and time condition defined after conversion.

The package converts a bounded 100,000-entry AMLNet sample into blockchain-style transfer entries while keeping the path back to the original source. For each converted transfer, it records the source dataset, source file, source entry, original account identifiers, and AMLNet category. The converter also maps source accounts to deterministic virtual blockchain addresses, keeps amount and timestamp fields, and adds placeholder asset and workload identifiers. The converted workload asks a narrower question: do checks over sender, receiver, amount, and time select entries similar to AMLNet’s categories?

For structuring and layering, the package compares AMLNet categories with simple checks whose field and threshold settings are chosen on one subset and evaluated on a held-out subset; the current held-out F1 values are about 0.49 and 0.48. For velocity and volume, direct overlap falls to 0.000 F1 and 0.015 F1, showing category-condition mismatch after conversion.

EX-Graph/Dune NFT sender-receiver workload. EX-Graph/Dune contributes sender-receiver links from NFT wash-trade data [12, 10]. The staged source has 22,702 entries and contains NFT name, token id, transaction hash, sender address, and receiver address. These fields make it useful for graph replay and back-and-forth transfer queries.

The staged source does not include price or block time fields. The package therefore treats this workload as sender-receiver graph data only. The converter assigns deterministic replay timestamps and unit amount placeholders so that the common transfer schema can process the data.

The source also illustrates why wash-trade identification and circular-trade identification remain separate. A source wash-trade category can support a back-and-forth NFT-transfer workload, but it does not define a separate circular-trade answer. The current graph summary reports 22,702 converted entries and 22,551 source-marked transactions; the same-token back-and-forth summary produces 3,379,930 matches across 886 users. These counts show that the sender-receiver graph is dense enough to stress graph processing.

The EX-Graph/Dune workload is therefore best used to test graph ingestion and bounded back-and-forth transfer queries. It does not test wash-trade volume or real-time detection latency because the staged source lacks price and block time.

Table 3. Public-source mapping by workload family. Each entry separates the public data used from the missing fields or answers that this package fills with deterministic generated data and explicit source information.

Workload family	Public data used	What the source misses	What the package adds
W1: Recent-history rules	AMLNet and other financial traces can anchor amount, time, and account distributions	Public traces do not contain service-specific account tier, verification status, or configured cap answers	Blockchain-style entries with generated rolling-window answers and replay summaries
W2: Graph patterns (NFT)	NFT sender-receiver data as one public graph-abuse case; NFT-70M market traces	EX-Graph/Dune has sender-receiver links but no prices or times; NFT-70M has market fields but no wash or circular-trade answers	Copied categories where available plus generated graph patterns for controlled recall
W2: Graph patterns (AML)	AMLNet transaction graph and categories from AMLNet	AMLNet is not native blockchain data, and its categories are provided by the source dataset rather than real-world legal findings	Deterministic virtual addresses, preserved copied-category information, and generated graph/path answers
W3/W5/W6: External-risk response	Sanctions-style feeds and incident seeds provide realistic external inputs	Feeds do not isolate delay, noise, which accounts are covered, or how far quarantine spreads	Controlled sweeps over risk-entry delay, false positives, false negatives, account coverage, and how far quarantine spreads
W4: Public-source translation	AMLNet, EX-Graph/Dune, and NFT-70M	Schemas, fields, and licenses differ across sources	Conversion scripts, source manifests, checksums, transfer-format fields, and notes on unsupported claims

NFT-70M market workload. NFT-70M provides the complementary NFT data source [13]: it has market fields with timestamps and prices, while EX-Graph/Dune provides sender-receiver links without price or block time. NFT-70M is released as a much larger market trace, and the local package stages seven bounded 1,000-entry samples from different offsets to capture variation across the upstream trace. Unlike the EX-Graph/Dune sender-receiver source, NFT-70M contains market fields such as transaction timestamp, price, USD price, seller account, buyer or winner account, contract address, token id, chain, and token metadata identifiers. These fields make it useful for ingestion, distribution checks, and historical-window pattern scans.

NFT-70M does not provide wash-trading answers in this package. The account and metadata identifiers are anonymized integer ids rather than public Ethereum addresses, so the package maps them to deterministic virtual addresses. The 7,000-entry sample contains no source wash-trading category. A scan over the original source fields found two same-asset back-and-forth cases within a one-day window, and denser activity when broadened from exact assets to collections. These observations check that the replay path preserves timestamps and prices; they are not precision or recall measurements.

The current sample is also useful for distribution checks. The staged sample spans multiple chains, with Ethereum and Polygon/Matic as large components and Solana also represented. The price distribution is broad: the scan over the original source fields reports a median USD price around 37 and a 95th percentile above 1,100, with a maximum above 128,000 in the bounded sample. This makes NFT-70M a useful complement to EX-Graph/Dune. EX-Graph/Dune gives sender-receiver categories from its source but no prices or times; NFT-70M gives prices and times but no wash-trading answer.

Common conversion method. All converted workloads follow the same conservative pattern. Source identifiers become deterministic virtual addresses. Source timestamps are preserved when available and replaced with deterministic replay time only when the source lacks time. Asset identifiers are retained when the source provides token or collection fields; replay-only placeholders are marked. Category entries remain separate from ordinary transfers and record whether the category came from the generator, a source dataset, an external feed, an event seed, a sender-receiver source, or no category at all. Result entries store the method, metric, config hash, and output path. The conversion files remain verbose because the source trail is part of the claim boundary.

The conversion process is intentionally deterministic. Re-running the same converter on the same source file should produce the same virtual addresses, workload ids, category entries, and source manifests. This makes the package auditable even when raw public source data is not redistributed in the default package. A reviewer can inspect the source manifest, download the upstream dataset under its own license, verify checksums, and rebuild the converted entries. A user who only wants to run the public package can start from the included generated data and summaries without accepting any new raw-data license.

5 Workload Package Evaluation

The previous sections define the workloads, source conversions, and category meanings. This section evaluates whether the released files preserve those meanings in checks that a user can inspect or rerun.

The section answers five practical questions. Are the release files present, and can the package scripts read them? Do the different trace families visibly differ? Can public-dataset inputs be rewritten into the common transfer format while preserving where each field came from? Can graph queries run over the converted sender-receiver data? Can delayed or noisy external risk entries be varied without changing the underlying transaction trace?

The package intentionally combines generated traces, converted AML-style transactions, NFT sender-receiver traces without prices, and NFT market traces with prices. The evaluation question is whether those differences remain visible when a user runs or inspects the package. Table 4 summarizes the checks and states what each check supports.

Table 4. Current package checks. Each check states the package question being tested, what the package can support, and what should not be inferred from that check.

Check	Inputs	What it establishes	What it does not establish
Release verification	Schemas, manifests, checksums, packaged files	The release files exist, match the schemas, and can be rebuilt from the release layout	Legal permission to redistribute upstream raw source data
Distribution check	Generated data, AMLNet translation, EX-Graph/Dune, bounded NFT-70M sample	The trace families visibly differ in amount, entity participation, and timing	Generated traffic matches production activity
Public conversion	AMLNet, EX-Graph/Dune, NFT-70M source fields	Entries from public datasets can be rewritten into the common transfer format while preserving where fields came from	That copied categories are legal ground truth
Graph-pattern scan	EX-Graph/Dune sender-receiver data and NFT-70M market data	Back-and-forth transfer queries can run over converted data	Every back-and-forth pattern is wash trading or market abuse
Delayed or incorrect external risk entries	Generated scenarios with external risk entries	Entry delay, entry errors, accounts covered, and quarantine spread can be varied separately	Real-world feed prevalence or accuracy

The distribution check is a quick check for whether the package mixes unlike traces while still telling the user that they are unlike. The package includes generated plots for transfer amount, entity participation, and inter-arrival time. Here, entity participation means how many replay entries include an entity as sender or receiver. Repeated transfers are counted repeatedly. This statistic therefore measures entry appearances, not the number of distinct counterparties. The p50 value in Table 5 is the median, and p95 is a high-end value: 95 percent of entries or entities are at or below that value.

Table 5. First distribution check. The package spans different regimes, which supports replay use and source-transparency checks.

Source input / trace	Entries	Amount p50	Amount p95	Entity participation p95
Generated workloads (temporal cap)	260	10.1	200.0	35.8
Generated workloads (NFT graph)	41	199.6	213.4	2.0
AMLNet	100,000	163.7	2,908.1	33.0
EX-Graph/Dune	22,702	1.0	1.0	81.0
NFT-70M	7,000	39.8	1,137.9	4.0
Total shown	130,003	—	—	—

These numbers are expected to differ across trace families. EX-Graph/Dune becomes a unit-amount sender-receiver workload because the staged source lacks price. NFT-70M has a broad price range but no wash-trading answer. AMLNet is much larger than the small generated examples. The distribution check therefore supports source transparency across unlike traces.

The graph checks ask whether converted data can support relationship-based queries. A back-and-forth transfer pattern means that account A sends an NFT or token to account B, and later account B sends the same NFT or token back to account A. EX-Graph/Dune supports sender-receiver graph queries without price or block-time claims. NFT-70M preserves timestamps and prices and supports same-asset scans, but has no wash-trading answer. The two NFT sources therefore test different parts of a graph task.

Table 6. Reference checks shipped with the package. Results are workload checks under the stated category meanings.

Workload/source group	Simple check	Scope	Result	What it shows
W1 Recent-history rules	Single-transfer amount threshold	W1 simple rolling cap	precision 0.167; recall 1.000; F1 0.286	Generated workloads
	Rolling-history reference	W1 simple rolling cap	precision 0.863; recall 1.000; F1 0.895	
	Rolling-history reference	W1 paired rolling cap	precision 0.186; recall 1.000; F1 0.313	
W2 Graph patterns	Same-asset back-and-forth graph rule	W2 back-and-forth pattern	precision 0.429; recall 1.000; F1 0.556	Generated workloads
W3/W5/W6 External-risk response	Direct block, no delay	risk response	target recall 0.287; precision 1.000	External-risk summaries
	2-hop quarantine, no delay		target recall 1.000; precision 0.078	
	Direct block, 168h delay		target recall 0.265; precision 1.000	
W4 Public-source translation	AMLNet structuring translation	W2 AMLNet structuring	held-out F1 0.488	AMLNet agreement
	AMLNet layering translation	W2 AMLNet layering	held-out F1 0.480	AMLNet agreement
	AMLNet velocity overlap	W1 AMLNet velocity	overlap F1 0.000	AMLNet mismatch
	AMLNet volume overlap	W1 AMLNet volume	overlap F1 0.015	AMLNet mismatch
	EX-Graph/Dune sender-receiver scan	W2/W4 NFT sender-receiver	3,379,930 same-token back-and-forth patterns	EX-Graph/Dune topology
	NFT-70M market scan	W2/W4 market trace	2 same-asset back-and-forth pairs; 53 same-collection pairs	NFT-70M unlabeled

Result labels. Generated workloads means controlled checks against answers created by the generator, not legal or market-abuse ground truth. External-risk summaries means controlled timing and response-coverage scenarios, not proof of incident prevention. AMLNet agreement and AMLNet mismatch compare AMLNet categories with checked conditions after conversion. EX-Graph/Dune topology means sender-receiver structure without price or timing claims. NFT-70M unlabeled means the trace has no answer for precision or recall.

The delayed-risk check fixes the transaction generator and varies five settings: risk-entry delay, false-positive entries, removed true entries, entity coverage, and quarantine radius. If a risk entry is available before the tested response, the transfer is in-time actionable. If the transfer has already occurred, the later entry supports only review or reporting. False positives and broader quarantine increase the accounts or transfers included in the response, including collateral review cases.

Reference package checks. The package includes reference checks so that users can run each workload family before replacing the method under test. These scripts verify that inputs, categories, and outputs preserve the meanings defined above.

Table 6 groups the current runs by workload family or source branch. W1 checks test whether the method has enough history for a time-window rule. W2 checks test replayed back-and-forth patterns. W3/W5/W6 checks vary external-risk delay and quarantine coverage. W4 checks compare only against categories or sender-receiver fields that the public source actually contains. The last column names the source input or support meaning.

The W1 Recent-history group shows why time-window tasks need the right history. A current-transfer threshold finds the injected positives but produces many false positives because it sees only the current transfer. A rolling-history reference improves the simple cap workload because it can see previous transfers in the window. The same reference remains weak for the paired cap workload because amount and time alone do not express the full account-pair condition.

The remaining groups expose different boundaries. W2 finds all generator-inserted graph positives but also flags broader back-and-forth patterns. W3/W5/W6 shows that broader quarantine recovers target recall while increasing collateral review, and that delayed risk entries convert earlier contacts into after-the-fact review. W4 shows moderate agreement with AMLNet structuring and layering categories but near-zero direct overlap for velocity and volume, indicating category-condition mismatch after conversion.

6 Discussion

The package evaluation leaves two release questions: which files can be distributed directly, and which public-source materials must remain as manifests, summaries, or rebuild instructions; and what users should not infer from generated answers, copied public categories, unlabeled market traces, or simple package checks.

Table 7. Release policy for the package. The package is designed to be useful without silently redistributing upstream raw source data whose licenses require separate handling.

Release status	Included files	Reason
Distributed directly	Generated transfer and category data, schemas, data card, notes on what the package does not claim, release scripts, generated distribution plots	These are project-generated files and support immediate workload use
Distributed as manifests and summaries	AMLNet, EX-Graph/Dune, and NFT-70M source notes, checksums, fetch or rebuild instructions, conversion scripts, and derived summaries	These preserve reproducibility while respecting upstream license boundaries without bundling raw source data
Not redistributed by default	Raw AMLNet, EX-Graph/Dune, and NFT-70M source data	Upstream datasets have non-commercial or share-alike terms; users should fetch them under their own accepted terms

6.1 Release Policy

The package includes a dataset card, a document listing what the package does not claim, citation metadata, workload organization, workload-source mapping, release layout, JSON schemas for transfers, categories, and results, generated distribution-check outputs, and simple-check outputs. For this compact package, the checked release policy is the manifest for public datasets and the package checker. The public dataset manifest verifies 3 external datasets and 22 files, and the compact package checker confirms that generated data is included. It also confirms that public raw source data that cannot be redistributed is excluded.

Table 7 summarizes what can be redistributed directly and what must remain as manifests or rebuild instructions. The distinction is explicit because the package mixes project-generated data with summaries derived from public sources.

The package policy follows from the category and source meanings above. Project-generated data and answers can be distributed directly after final license review because they are produced by this project. AMLNet, EX-Graph/Dune, and NFT-70M are different. They are external sources with non-commercial or share-alike terms. Entries converted from those public datasets are therefore not redistributed by default. Instead, the package ships source manifests, checksums, download instructions, conversion scripts, and derived summaries. A user who obtains the upstream datasets under their original licenses can verify the raw files and rerun the converter to regenerate the same transfer-format entries. The regenerated outputs store the source dataset, source file, source-entry number, and original endpoint identifiers. That metadata keeps the path from upstream source data to the converted entry auditable. The package can do this without bundling raw upstream source data.

The verification step checks source manifests, checksums, and generated files. The public release is designed so that a user can inspect the schemas, run the simple checks, and understand what each category can and cannot mean without downloading upstream raw source data whose license terms require separate handling.

The public release includes generator configs and generated category entries, source manifests and checksums, conversion scripts, simple-check scripts, generated summaries and plots, and a verification script for local paths, checksums, and schema conformance.

6.2 Limitations

The package has three limits. First, it does not decide legal compliance: generated answers are exact only for the generated rule, copied public categories remain approximate evidence, and unlabeled traces support ingestion and distribution checks rather than detection accuracy. Second, public datasets help ground the workload shapes, but the current release does not prove that generated traffic matches production blockchain activity. Third, external raw datasets have non-commercial or share-alike terms, so public releases may need manifests and rebuild instructions rather than raw source rows. These limits keep the package useful for controlled replay and claim-boundary testing without turning its categories into legal findings or prevalence estimates.

7 Conclusion

Regulatory monitoring systems on blockchains cannot be evaluated with a flat transaction file alone. They need workloads that state what evidence is available, where each category came from, when the evidence arrived, and which response the system is being asked to support. This paper introduced six workload families for that setting: recent-history rules, repeated-transfer graph patterns, delayed external risk entries, public-source translation, response footprint, and evidence freshness.

We also built a reproducible dataset package for those workloads by combining generated test cases with converted AMLNet, EX-Graph/Dune, and NFT-70M samples in a common replay format. The deterministic reference checks show why the separation matters: missing history changes rolling-rule results, public categories can mismatch checked conditions, topology-only traces cannot support price or timing claims, and delayed risk entries expose response-scope tradeoffs. The result is an auditable benchmark unit for comparing regulatory monitoring methods while keeping evidence, category source, timing, and response separate. The package makes these boundaries explicit, giving researchers, protocol teams, and compliance operators a common target for improving regulatory monitoring systems before those systems are connected to production policy decisions.

References

- [1] Financial Action Task Force. Updated Guidance for a Risk-Based Approach to Virtual Assets and Virtual Asset Service Providers. FATF, 2021. <https://www.fatf-gafi.org/en/publications/Fatfrecommendations/Guidance-rba-virtual-assets-2021.html>.
- [2] U.S. Department of the Treasury, Office of Foreign Assets Control. Publication of Sanctions Compliance Guidance for the Virtual Currency Industry and Updated Frequently Asked Questions. 2021. <https://ofac.treasury.gov/recent-actions/20211015>.
- [3] U.S. Department of the Treasury. Treasury Releases 2023 DeFi Illicit Finance Risk Assessment. 2023. <https://home.treasury.gov/news/press-releases/jy1391>.
- [4] U.S. Department of the Treasury. Treasury Releases First Ever Non-fungible Token Illicit Finance Risk Assessment. 2024. <https://home.treasury.gov/news/press-releases/jy2382>.
- [5] Financial Action Task Force. Virtual Assets Red Flag Indicators of Money Laundering and Terrorist Financing. FATF, 2020. <https://www.fatf-gafi.org/en/publications/Methodsandtrends/Virtual-assets-red-flag-indicators.html>.
- [6] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics. arXiv:1908.02591, 2019.
- [7] Peilin Zheng, Zibin Zheng, and Hong-ning Dai. XBlock-ETH: Extracting and Exploring Blockchain Data From Ethereum. arXiv:1911.00169, 2019.
- [8] Claudio Bellei, Muhua Xu, Ross Phillips, Tom Robinson, Mark Weber, Tim Kaler, Charles E. Leiserson, Arvind, and Jie Chen. The Shape of Money Laundering: Subgraph Representation Learning on the Blockchain with the Elliptic2 Dataset. arXiv:2404.19109, 2024.
- [9] Cuneyt Gurcan Akcora, Yitao Li, Yulia R. Gel, and Murat Kantarcioglu. BitcoinHeist: Topological Data Analysis for Ransomware Detection on the Bitcoin Blockchain. arXiv:1906.07852, 2019.
- [10] Dune. nft.wash_trades documentation. <https://docs.dune.com/data-catalog/curated/nft-trades/evm/nft-wash-trades>. Accessed May 2026.
- [11] Sabin Huda, Ernest Foo, Zahra Jadidi, MA Hakim Newton, and Abdul Sattar. AMLNet: A Knowledge-Based Multi-Agent Framework to Generate and Detect Realistic Money Laundering Transactions. arXiv:2509.11595, 2025. Dataset entry: <https://zenodo.org/records/16736515>.
- [12] Qian Wang, Zhen Zhang, Zemin Liu, Shengliang Lu, Bingqiao Luo, and Bingsheng He. EX-Graph: A Pioneering Dataset Bridging Ethereum and X. International Conference on Learning Representations, 2024. Public repository: <https://github.com/Persdre/EX-Graph>.
- [13] Davide Costa, Lucio La Cava, and Andrea Tagarelli. Unraveling the NFT economy: A comprehensive collection of Non-Fungible Token transactions and metadata. *Data in Brief*, 51:109749, 2023. Dataset page: https://huggingface.co/datasets/MLNTeam-Unical/NFT-70M_transactions. Accessed May 2026.

A Detailed Source-to-Workload Graph

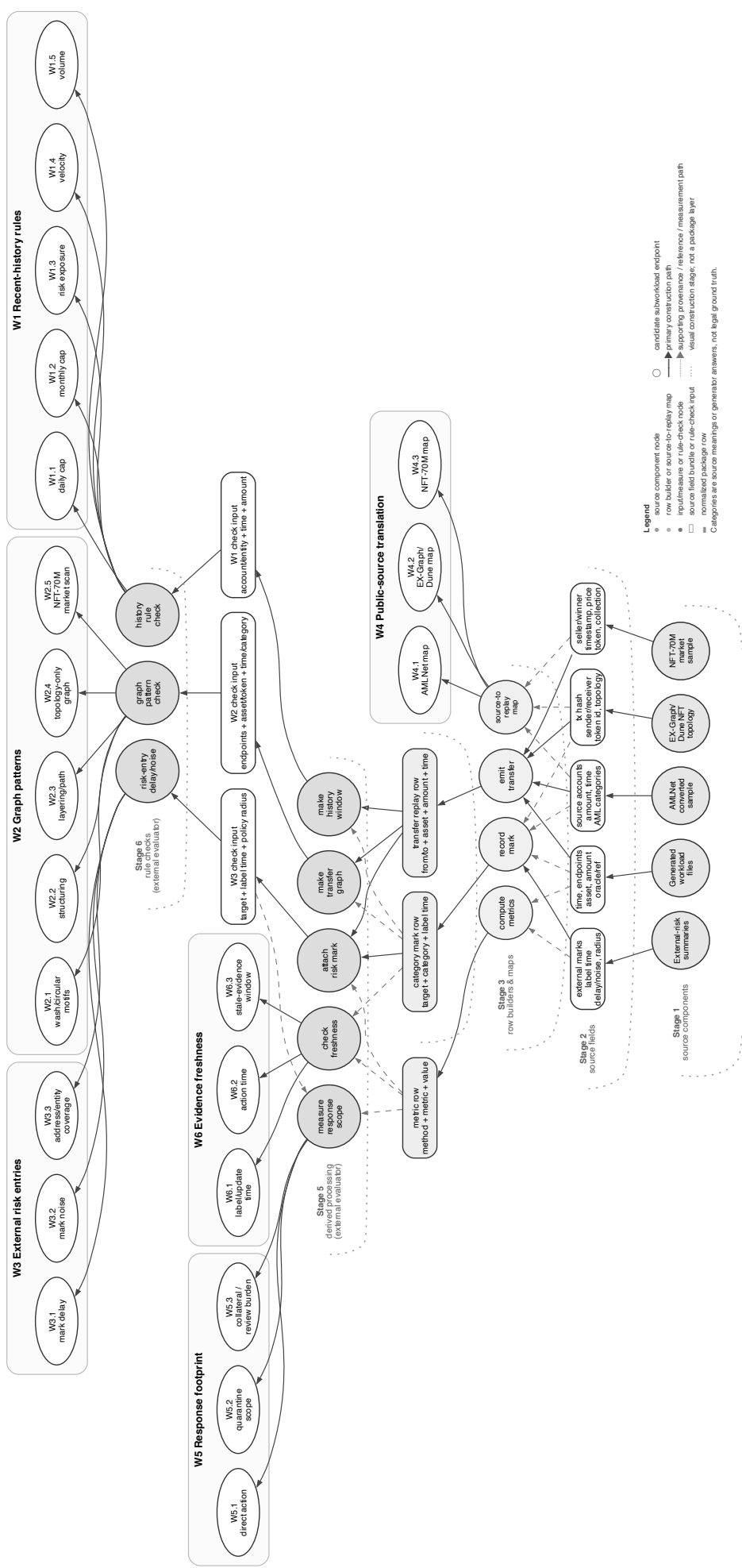


Figure 2: Detailed source-to-workload construction graph. This is an audit view of the current package, not a second taxonomy.